TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

BÁO CÁO THỰC HÀNH

Bài thực hành số 1: Tính toán Bits

Môn học: Lập trình Hệ thống

Lớp: NT209.R11.ANTN

THÀNH VIÊN THỰC HIỆN (Nhóm 10):

Họ và tên	MSSV
Đỗ Bách	25520110
Đinh Nguyễn Tuấn Đạt	25520260
Đinh Nguyễn Bảo Anh	25520042

ĐÁNH GIÁ KHÁC:

Tổng thời gian thực hiện	3 ngày
Phân chia công việc	
Ý kiến *(nếu có)* + Khó khăn + Đề xuất, kiến nghị	

Phần bên dưới của báo cáo này là báo cáo chi tiết của nhóm thực hiện

MỤC LỤC

A. BÁO CÁO CHI TIẾT **2**
1. Tính toán bit đơn giản **2**
a. Câu 1.1: negative(int x) 2
b. Câu 1.2: cal100x(int x) 3
c. Câu 1.3: flipByte(int x, int n) 4
d. Câu 1.4: getnbit(int x, int n) 5
e. Câu 1.5: round2n(int x, int n) 7
2. Các phép kiểm tra dựa trên tính toán bit **9**
a. Câu 2.1: isSameSign(int x, int y) 9
b. Câu 2.2: isPositive(int x) 10
c. Câu 2.3: isMulpw2(int x, int n) 11
d. Câu 2.4: isLess2n(int x, int n) 12
e. Câu 2.5: logicNot(int x) 15
B. TÀI LIỆU ĐÍNH KÈM **17**

A. BÁO CÁO CHI TIẾT

1. Tính toán bit đơn giản

a. Câu 1.1: negative(int x)

Đề bài

1.1	int **negative**(x)	Tính giá trị của **–x** không dùng dấu – • Input: Số nguyên x • PT: **negative**(0)==0 && **negative**(9)==-9 && **negative**(-5)==5	1	5

Ý tưởng: sử dụng kỹ thuật lấy bù 2 của một số dương để lưu số âm của chính số dương đó trong máy tính. Phép lấy bù 2 có tính đối hợp, tức là bù2(bù2(x)) = x nên mọi số nguyên sau khi lấy bù 2 sẽ trở thành số đối của nó. Có thể kiểm chứng như bên dưới

Với số nguyên X bất kì có biểu diễn bit là $\overline{x_{w-1}x_{w-2}\dots x_0}$, ta có

$$B2T(X) = -x_{w-1}.2^{w-1} + \sum_{i=0}^{w-2} x_i.2^i$$

$$B2T(\sim X + 1) = -\overline{x_{w-1}}.2^{w-1} + \sum_{i=0}^{w-2} \overline{x_i}.2^i + 1$$ ở đó $\overline{x}$ là đảo bit của x

$$B2T(\sim X + 1) + B2T(X) = -\overline{x_{w-1}}.2^{w-1} + \sum_{i=0}^{w-2}\overline{x_i}.2^i + 1 - x_{w-1}.2^{w-1} + \sum_{i=0}^{w-2} x_i.2^i$$

$$-2^{w-1} + \sum_{i=0}^{w-2} 2^i + 1 = \overline{11\ldots1}_2 + 1 = 0$$

Do đó ta có logic hàm negative(x) là $\sim x + 1$

```
2   //1.1
3   int negative(int x)
4   {
5       return ~x + 1;
6   }
```

Kết quả chạy code

```
1.1 negative            Pass.
1.2 cal100x                     Pass.
1.3 flipByte            Pass.
1.4 getnbit                     Pass.
1.5 round2n                     Pass.
2.1 isSameSign          Pass.
2.2 isPositive          Pass.
```

b. Câu 1.2: cal100x(int x)

Đề bài

1.2	int **cal100x**(x)	Tính giá trị 100*x mà không dùng dấu *. • Input: Số nguyên x (giới hạn sao cho 100x biểu diễn đủ trong số có dấu 4 byte) • PT: **cal100x**(0)==0 && **cal100x**(9)== 900 && **cal100x**(-5)==-500	1	10

Bài này vì không được thực hiện phép toán nhân * nhưng thực tế vẫn tồn tại phép nhân hạn chế được biết đến với tên gọi left shift - nhân với lũy thừa của 2. Cụ thể

$$x * 2^n = x << n$$

Ta tách 100 thành tổng các lũy thừa của 2 rồi tiến hành left shift

$$100 * x = (64 + 32 + 4) * x = 2^6 * x + 2^4 * x + 2^2 * x$$

$$= (x << 6) + (x << 5) + (x << 2)$$

```
8   //1.2
9   int cal100x(int x){
10      return (x << 6) + (x << 5) + (x << 2);
11  }
```

Kết quả chạy code

```
1.1 negative            Pass.
1.2 cal100x                     Pass.
1.3 flipByte            Pass.
1.4 getnbit                     Pass.
1.5 round2n                     Pass.
2.1 isSameSign          Pass.
2.2 isPositive          Pass.
```

c. Câu 1.3: flipByte(int x, int n)

Đề bài

1.3	int **flipByte**(x,n)	Lật 1 Byte thứ n của số nguyên **x** *(các byte được đánh thứ tự từ 0 đến 3 từ phải sang trái)*, từ bit 0 thành 1 và ngược lại. • Input: Số **x** nguyên, số **n** (0 ≤ n ≤ 3) • PT: **flipByte**(10,0)==245 && **flipByte**(0,1)== 65280 && **flipByte**(0x5501,1)==0xaa01	2	10

Ý tưởng là sẽ tạo ra một mask theo n để thực hiện xor với x

Cơ sở lý thuyết:

$$x \oplus 0 = x$$

$$x \oplus 1 = \sim x$$

Do đó ta cần tạo mask là $0x0000FF00$ với FF là vị trí byte thứ n theo thứ tự tăng từ phải sang trái.

Vì 1 byte = 8 bit nên ta có thể nghĩ đến left shift $(<< 8)$ hoặc $(<< 16)$ hoặc $(<< 24)$

Đầu tiên, để FF bắt đầu ở vị trí byte thứ n nghĩa là ta đã left shift $8 * n$ bit. Mà $8 * n$ có thể được viết ở dạng left shift là $n << 3$. Do đó ta có biểu thức đầy đủ bằng left shift để tạo ra mask như sau

$$mask = 0xff << (n << 3)$$

Cuối cùng đem x xor với mask ta sẽ được kết quả phép đảo byte thứ n

$$flipByte(x, n) = x \oplus (0xff << (n << 3))$$

```
13  //1.3
14  int flipByte(int x, int n) {
15      return (0xff << (n << 3)) ^ x;
16  }
```

Kết quả chạy code

```
1.1 negative            Pass.
1.2 cal100x                     Pass.
1.3 flipByte            Pass.
1.4 getnbit                     Pass.
1.5 round2n                     Pass.
2.1 isSameSign          Pass.
2.2 isPositive          Pass.
```

d. Câu 1.4: getnbit(int x, int n)

Đề bài

		65280 && **flipByte**(0x5501,1)==0xaa01		
1.4	int **getnbit**(x,n)	Trả về n bit thấp nhất (bên phải) của số nguyên x (*các bit được đánh số thứ tự từ 0 đến 31 từ phải sang trái).* • Input: Số **x** nguyên, số **n** (0 ≤ n ≤ 31) • PT: **getnbit**(15,3)==7 && **getnbit**(63,6)==63 && **getnbit**(30,2)==2 **Nâng cao:** Trả về kết quả với mọi giá trị n không âm (0 ≤ n), tối đa trả về 32 bit. **PT:** getnbit(10, 32) == 10 && getnbit(-50, 35) == -50	2/**3**	10/**25**

Ý tưởng làm bài:

- Để lấy được n bit cuối cùng, ta chỉ cần and nó với n bit 1 ở cuối !

```
44      // x = x & full_one;
45
46
47      int one_big_num = (1 << n) ;
48
49
50
51      int full_one = one_big_num + (~1+1) ;
52
53
54      x = x & full_one;
55
56
57      return x;
```

Ví dụ, nếu muốn lấy 3 bit cuối của 1011 ta chỉ cần and nó với 111

- Ta có đoạn code sau
- Giải thích code:
- one big num là biến đã được dịch sang n biến ! nếu n là 3 thì sau phép ấy, ta sẽ có 1 số có bit 1000
- Khi lấy one bit num trừ đi 1, ta sẽ co1 đúng n số 1 mà ta cần để giải (1000 - 1 = 111)
- khi ấy, chỉ cần xor xong là được !

```
 gnomeright  ~/PARA/Area/HOC_TREN_TRUONG/pwn/buoi1 >> gcc -o main main.c
 gnomeright  ~/PARA/Area/HOC_TREN_TRUONG/pwn/buoi1 >> ./main
1.1 negative            Pass.
1.2 cal100x                     Pass.
1.3 flipByte            Pass.
1.4 getnbit                     Pass.
```

Tuy nhiên, với đề advance, ta yêu cầu sẽ phải lấy hết, kể cả khi cái số n nó lớn hơn là 32 !

mà kiểu khi ấy, với kiểu int thì sẽ bị overflow và quay ngược ! (nếu n = 35 và ta sử dụng mẹo dịch bit như trên thì sẽ bị ra kết quả giống với n = 2)

Nên em có đoạn code này

```
unsigned int getnbit(unsigned int x, int n) {
    int overflow_padd = 0 + ~(n >> 5) + 1;

    int one_big_num = (1 << n) ;

    int full_one = one_big_num + (~1+1) ;

    full_one = full_one | overflow_padd;

    x = x & full_one;
}
```

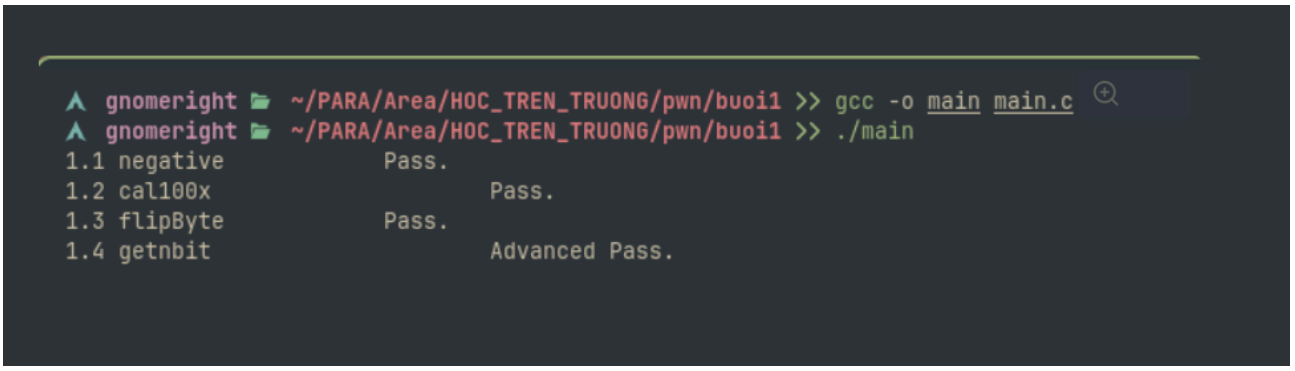
ta sẽ kiểm tra số ncó lớn hơn 32 ? nếu lớn hơn thì coi như full 1 hết (nghĩa là lấy hết
cả số int đấy) nếu nhỏ hơn thì chỉ lấy n bit

overflow_padd = 0 + ~(n >> 5) + 1
phép n >> 5 là n / 32 lấy nguyên, nếu phần nguyên là 0 thì 0-0 = 0 (các bit đều 0 hết)
còn nếu phần nguyên là 1 thì 0 - 1 = -1 (các bit đều là 1)

khi ấy, ta lấy bộ lọc này, or với lại cái full_one ở trên !

nếu -1 thì sẽ cho ra full 32 bit 1 hết, nếu 0 thì or với full_one sẽ là chính full one !

sau đó chạy và ta có đáp án



e. Câu 1.5: round2n(int x, int n)

Đề bài

		PT: getnbit(10, 32) == 10 && getnbit(-50, 35) == -50		
1.5	int **round2n(x,n)**	Làm tròn xuống x đến số bội của 2^n gần nhất. • Input: Số x, n nguyên và không âm. • PT: **round2n**(10,3) == 8 && **round2n**(16,2) ==16 && **round2n**(123,6) == 64 **Nâng cao:** Làm tròn số x lên hoặc xuống, tùy vào giá trị của nó lớn hay nhỏ hơn ½ khoảng giá trị. **PT:** round2n(15,3)==16 && round2n(20,5)==32	2/**3**	10/**30**

ý tưởng làm :

- Ta sẽ có cách làm tròn xuống bội của 2^n bằng cách chia lấy nguyên. Coi số U là phần nguyên khi ta chia x cho 2^n
- KHi ấy, U*2^n là một bội của 2^N (Làm tròn xuống)
- Mở rộng ý tưởng ra. Ta sẽ làm tròn số lên bằng cách nào ? Giữ nguyên cơ chế làm tròn xuống Ta sẽ cộng với 1 nửa của 2^n, khi ấy, nếu phần dư của x sau khi chia 2^n đã lớn hơn 0.5 sẽ được đẩy lên hơn 1 luôn (và làm tròn xuống như trước)

Ta có đoạn code sau:

```c
}

//1.5
int round2n(int x, int n){
    int boi_n = 1 << n;
    int pad_them = boi_n >>  1  ;

    x += pad_them;

    int chia_nguyen = x >> n ;

    x = chia_nguyen << n ;

    return x;
}
```

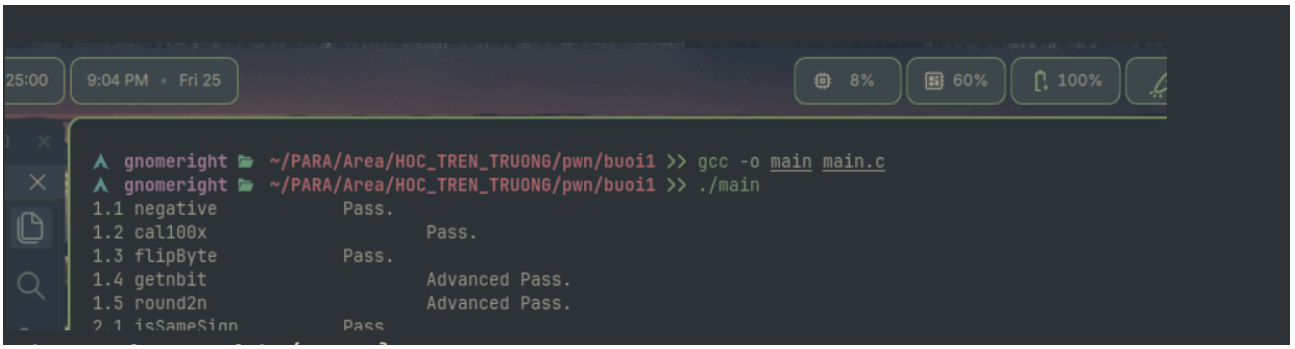
boi_n = 1 << n chính là số 2^n để ta tính toán

pad_them = boi_n >> 1 (là số 2^n - 1)

x += pad_them

khi ấy, chia lấy nguyên (làm tròn xuống) sẽ được đưa về kết quả gần nhất

Chạy và ta có kết quả advanced pass



2. Các phép kiểm tra dựa trên tính toán bit

a. Câu 2.1: isSameSign(int x, int y)

2.1	int **isSameSign**(x,y)	Kiểm tra 2 số x và y có cùng dấu (cùng âm hoặc cùng không âm). Trả về 1 nếu x và y cùng dấu, 0 nếu ngược lại. • Input: Số nguyên x, y • PT: **isSameSign**(2,-2)==0 && **isSameSign**(-5,10)==0 && **isSameSign**(0,16)==1 && **isSameSign**(-4,-20)==1	1	8

Ý tưởng là ta sẽ kiểm tra bit MSB của mỗi số x, y để check dấu của mỗi số. Ta mong muốn lấy ra dấu của mỗi số ở dạng:

Số dương: $\overline{00\ldots 00}_2$

Số âm: $\overline{00\ldots 01}_2$

Sau đó thực hiện phép xor các dạng số âm/dương đó với nhau, lấy kết quả đó xor với 1 để đảo bit cho đúng yêu cầu đề bài (cùng dấu ra 1, trái dấu ra 0).

Bằng cách right shift $(>> 31)$ ta lấy ra bit đầu của một số. Tuy nhiên có một vấn đề:

Nếu $x \geq 0$ thì $(x >> 31) = \overline{00\ldots}_2$. Nếu $x < 0$ thì $(x >> 31) = \overline{11\ldots 1}_2$

Với $x < 0$ kết quả trả về là 32 bit 1, chưa đúng với mong muốn, nên ta sẽ thực hiện $\& 1$ vào kết quả dịch bit đó để lấy ra đúng định dạng mong muốn

Vậy tóm lại $MSB(x) = (x >> 31) \& 1$

Như vậy, biểu thức hoàn chỉnh để thực hiện yêu cầu là

$isSameSign(x, y) = MSB(x) \oplus MSB(y) \oplus 1 = ((x >> 31) \& 1) \oplus ((y >> 31) \& 1) \oplus 1$

Phép $\oplus\ 1$ ở cuối là để đảo kết quả về cùng dấu ra 1, trái dấu ra 0

```
29  //2.1
30  int isSameSign(int x, int y) {
31      return (((x >> 31) & 1) ^ ((y >> 31) & 1)) ^ 1;
32  }
```

Kết quả chạy code

```
2.1 isSameSign          Pass.
2.2 isPositive          Pass.
```

b. Câu 2.2: isPositive(int x)

2.2	int **isPositive**(x)	Trả về 1 nếu x dương (x > 0) • Input: Số nguyên x • PT: **isPositive**(10)==1 && **isPositive**(-5)==0 && **isPositive**(0)==0	2	10

Ý tưởng bài này tương tự bài **2.1** là kiểm tra bit MSB của số đó, nhưng đầu vào là số 0 và số âm phải trả về kết quả 0, chỉ có số dương trả về 1, nên bài này ta sẽ sử dụng số đối của số x để kiểm tra bit

Vì số dương thì MSB = 0, số 0 thì MSB = 0, số âm thì MSB = 1 kết quả "hơi ngược" với mong muốn là số dương trả về 1, số 0 thì trả về 0, số âm thì trả về 0. Do đó ta lấy bù 2 để MSB(số dương) = 1, MSB(số 0 và số âm) = 0. Lợi dụng việc số 0 luôn bất biến qua phép lấy bù 2

Cụ thể

	x	$MSB(x)$	$\sim x + 1$	$MSB(\sim x + 1)$
> 0	$\overline{0\ldots}_2$	0	$\overline{1\ldots}_2$	1
< 0	$\overline{1\ldots}_2$	1	$\overline{0\ldots}_2$	0
$= 0$	$\overline{00\ldots 0}_2$	0	$\overline{00\ldots 0}_2$	0

Có thể thấy $MSB(\sim x + 1)$ hoàn toàn trùng khớp với logic đầu ra của hàm **isPositive(int x)**. Do đó ta sử dụng $MSB(\sim x + 1)$ làm định nghĩa logic

Từ bài **2.1** ta cũng biết $MSB(\sim x + 1) = ((\sim x + 1) >> 31) \,\&\, 1$

Vậy biểu thức hoàn chỉnh của hàm **isPositive(int x)** là

$isPositive(x) = ((\sim x + 1) >> 31) \,\&\, 1$

Kết quả chạy code

```
2.1 isSameSign          Pass.
2.2 isPositive          Pass.
```

c. Câu 2.3: `isMulpw2(int x, int n)`

2.3	int **isMulpw2**(x,n)	Kiểm tra 1 số nguyên **x** có chia hết cho cho 2^n hay không. Trả về 1 nếu là chia hết, không thì trả về 0. • Input: Số nguyên **x** tùy ý, số nguyên **n** không âm. • PT: **isMulpw2**(16,4)==1 && **isMulpw2**(10,2)==0 && **isMulpw2**(48,3)==1	2	15

Yêu cầu bài toán

- **Đặc tả:** Kiểm tra xem số nguyên x có chia hết cho 2^n hay không ($x \bmod 2^n = 0$). Trả về `1` nếu chia hết, ngược lại trả về `0`.
- **Đầu vào:** Số nguyên x bất kỳ, số nguyên không âm n ($n \geq 0$).
- **Giới hạn:** Tối đa 15 toán tử (Max Ops: 15).

Phân tích và ý tưởng giải thuật

Ý tưởng chính của bài này là tận dụng đặc điểm biểu diễn nhị phân để kiểm tra xem `x` có chia hết cho 2^n hay không, thay vì sử dụng phép chia hoặc phép lấy dư.

Trong hệ nhị phân (biểu diễn bù 2), số x chia hết cho 2^n khi và chỉ khi phần dư trong phép chia x cho 2^n bằng 0. Phần dư này tương ứng chính xác với **n bit trọng số thấp nhất (LSB)** của x. Lý do là **n bit thấp nhất** chính là phần quyết định giá trị của số khi xét theo modulo 2^n, hay nói cách khác, chúng biểu diễn phần dư khi chia cho 2^n.

Để kiểm tra n bit cuối có toàn bộ bằng 0 hay không:

1. **Tạo mask:** Cần một giá trị có n bit cuối bằng `1` (giá trị $2^n - 1$). Do không được dùng toán tử trừ `-`, ta sử dụng biểu diễn bù hai: $2^n - 1 \equiv 2^n + (-1) = (1 \ll n) + \sim 0$ (vì $\sim 0 = -1$).
2. **Trích xuất n bit cuối:** Thực hiện phép `AND` bitwise giữa x và mask: `x & mask`.
3. **Chuẩn hóa kết quả:** Nếu x chia hết cho 2^n, giá trị `x & mask` bằng 0. Dùng toán tử `!` để đảo bit: `!(0) = 1` và `!(khác 0) = 0`.

Như vậy, toàn bộ thuật toán chỉ cần ba bước: tạo mask $2^n - 1$, dùng `AND` để lấy `n` bit thấp nhất, rồi dùng `!` để kiểm tra chúng có bằng 0 hay không.

Mã nguồn

```
int isMulpw2(int x, int n) {
  int mask = (1 << n) + ~0;
  x = !(x & mask);
```

```
  return x;
}
```

```
47  // 2.3
48  int isMulpw2(int x, int n) {
49      int mask = (1 << n) + ~0;
50      x = !(x & mask);
51      return x;
52  }
```

```
[Running] cd "/home/DntdToM/scripts/ThucHanhLTHT/Week01/" && gcc source.c -o source && "/home/DntdToM/scripts/ThucHanhLTHT/Week01/"source
1.1 negative        Pass.
1.2 cal100x         Pass.
1.3 flipByte        Pass.
1.4 getnbit         Failed.
1.5 round2n         Failed.
2.1 isSameSign      Pass.
2.2 isPositive      Pass.
2.3 isMulpw2        Pass.
2.4 isLess2n        Pass.
2.5 logicNot        Pass.
-------
Your score: 7.0
[Done] exited with code=0 in 0.456 seconds
```

d. Câu 2.4: `isLess2n(int x, int n)`

2.4	int **isLess2n**(x,n)	Trả về 1 nếu $x < 2^n$ • Input: • Số **x** nguyên dương • Số **n** ($0 \leq n \leq 30$) • PT: **isLess2n**(15,1)==0 && **isLess2n**(8,3)==0 && **isLess2n**(12,4)==1 && **isLess2n**(63,6)==1	2	15

Yêu cầu bài toán

- **Đặc tả:** Trả về 1 nếu $x < 2^n$, ngược lại trả về 0.
- **Đầu vào:** Số nguyên dương x ($x > 0$), số nguyên n ($0 \leq n \leq 30$).
- **Giới hạn:** Tối đa 15 toán tử (Max Ops: 15).

Phân tích và ý tưởng giải thuật

Cách tiếp cận 1: Xét dấu hiệu số (Cách được dùng trong bài)

Để kiểm tra $x < 2^n$, ta có thể chuyển bài toán về kiểm tra dấu của hiệu $x - 2^n$. Nếu hiệu âm thì $x < 2^n$; nếu hiệu không âm thì $x >= 2^n$.

- Xét bất đẳng thức: $x < 2^n \iff x - 2^n < 0$.

- Do cấm toán tử `-`, ta biểu diễn -2^n qua số đối bù hai: $-2^n = \sim(2^n) + 1 = \sim(1 \ll n) + 1$.
  Trong biểu diễn nhị phân, dịch số `1` sang trái `n` vị trí sẽ tạo ra 2^n. Vì vậy `1 << n` chính là 2^n.
  Cụ thể:
  1 << 0 = 00000001 = 1
  1 << 1 = 00000010 = 2
  1 << 2 = 00000100 = 4
  1 << 3 = 00001000 = 8
- Tính hiệu: $tmp = x + \sim powerOfTwo + 1$.
- Sau khi có hiệu $tmp = x - 2^n$, ta không cần biết chính xác giá trị của hiệu, mà chỉ cần biết nó âm hay không. Với số nguyên có dấu 32 bit, bit cao nhất (`MSB`), tức bit thứ 31, là bit dấu; vậy ta cần dịch bit sang phải `tmp >> 31` để đưa bit dấu từ vị trí 31 về vị trí thấp nhất (`LSB`):
  - Nếu $x < 2^n \implies tmp < 0 \implies$ bit thứ 31 là `1`. Phép dịch phải số học `tmp >> 31` sẽ cho kết quả là `-1` (chuỗi 32 bit `1`). Khi đó `(-1) & 1 = 1`.
  - Nếu $x \geq 2^n \implies tmp \geq 0 \implies$ bit thứ 31 là `0`. Phép dịch phải `tmp >> 31` ra `0`. Khi đó `0 & 1 = 0`.
- Do $x \in [1, 2^{31} - 1]$ và $2^n \in [1, 2^{30}]$, hiệu $x - 2^n$ không bao giờ bị tràn số nguyên 32-bit có dấu.

Tóm lại, ta tính $tmp = x - 2^n$ bằng biểu diễn bù 2. Nếu $x < 2^n$ thì `tmp` âm, bit dấu bằng `1`, nên `tmp >> 31` cho `-1`, sau đó `& 1` cho kết quả `1`. Ngược lại, nếu $x >= 2^n$ thì `tmp` không âm, phép dịch cho `0`, và kết quả cuối cùng là `0`.

Cách tiếp cận 2: Kiểm tra tiền tố bit (Có thêm vào Comment trong code)

Tuy nhiên, với điều kiện đề bài cho $x > 0$ và $0 \leq n \leq 30$, ta có thể giải quyết bài toán đơn giản hơn mà không cần thực hiện phép trừ.
Nếu $x < 2^n$, thì tất cả các bit từ vị trí `n` trở lên của `x` đều bằng `0`.
VD:
x = 5 = 00000101
n = 3
x >> 3
00000101 >> 3
00000000
Ngược lại, nếu $x >= 2^n$, thì chắc chắn tồn tại ít nhất một bit từ vị trí `n` trở lên bằng `1`, nên `x >> n` sẽ khác `0`.

- Vì $x > 0$ và $0 \leq n \leq 30$: Nếu $x < 2^n$, toàn bộ các bit từ vị trí n trở lên đều bằng `0`. Do đó phép dịch phải `x >> n` chắc chắn bằng `0`.
  VD:
  x = 5 = 00000101
  n = 3

x >> 3
00000101 >> 3
00000000

- Ngược lại, nếu $x \geq 2^n$ thì bit thứ n trở lên có ít nhất một bit 1, dẫn tới x >> n ≥ 1.
  VD:
  x = 10 = 00001010
  n = 3
  x >> 3
  00001010 >> 3
  00000001
- Vì vậy: $x < 2^n \iff !(x \gg n)$.

Cách 1 kiểm tra bằng dấu của $x - 2^n$, còn cách 2 nhìn trực tiếp vào các bit có trọng số từ 2^n trở lên. Vì x dương, chỉ cần các bit này đều bằng 0 là đủ kết luận $x < 2^n$.

Mã nguồn

```
int isLess2n(int x, int n) {
    // Cách 1:
    int powerOfTwo = 1 << n;
    int tmp = x + ~powerOfTwo + 1;
    x = (tmp >> 31) & 1;
    return x;
    /*
    Cách 2:
    x = x >> n;
    x = !x
    return x;
    */
}
```

```c
// 2.4
int isLess2n(int x, int n) {
    // Cách 1:
    int powerOfTwo = 1 << n;
    int tmp = x + ~powerOfTwo + 1;
    x = (tmp >> 31) & 1;
    return x;
    /*
    Cách 2:
    x = x >> n;
    x = !x
    return x;
    */
}
```

```
[Running] cd "/home/DntdToM/scripts/ThucHanhLTHT/Week01/" && gcc source.c -o source && "/home/DntdToM/scripts/ThucHanhLTHT/Week01/"source
1.1 negative        Pass.
1.2 cal100x         Pass.
1.3 flipByte        Pass.
1.4 getnbit         Failed.
1.5 round2n         Failed.
2.1 isSameSign      Pass.
2.2 isPositive      Pass.
2.3 isMulpw2        Pass.
2.4 isLess2n        Pass.
2.5 logicNot        Pass.
———
Your score: 7.0
[Done] exited with code=0 in 0.456 seconds
```

e. Câu 2.5: `logicNot(int x)`

2.5	int **logicNot**(x)	Trả về kết quả !x mà không dùng phép !. • Input: Số **x** nguyên (trong giới hạn int) • PT: **logicNot**(0)==1 && **logicNot**(1)==0 && **logicNot**(-15)==0 && **logicNot**(-2147483648) == 0	3	25

Yêu cầu bài toán

- **Đặc tả:** Mô phỏng toán tử logic `!x` mà không sử dụng `!`.
- **Đầu vào:** Số nguyên x bất kỳ (trong phạm vi 32-bit `int`).
- **Giới hạn:** Tối đa 25 toán tử (Max Ops: 25).

Phân tích và ý tưởng giải thuật

Ở câu này, yêu cầu là mô phỏng hành vi của toán tử logic `!`, nhưng bản thân toán tử `!` lại không được phép sử dụng. Vì vậy, trước tiên ta cần xác định chính xác `!x` hoạt động như thế nào.

Theo bảng chân trị logic:

- Nếu $x = 0 \implies$ kết quả trả về của `!x` là `1`.
- Nếu $x \neq 0 \implies$ kết quả trả về của `!x` là `0`.

Đặc trưng bit dấu trong hệ bù hai:

- Với $x = 0$: Giá trị đối của nó $-0 = 0$. Cả x và -x đều có bit dấu (bit 31) bằng `0`.
- Với $x > 0$: Số đối -x < 0 nên bit dấu của -x luôn là `1`.
- Với $x < 0$ (kể cả trường hợp biên $x = -2^{31} = 0\text{x}80000000$): Bản thân x đã có bit dấu là `1`.

Xây dựng thuật toán:

- Biểu diễn số đối $-x = \sim x + 1$ ($\rightarrow$ trong bù 2, `~x + 1` chính là `-x`).
- Bây giờ ta thực hiện phép OR giữa `x` và `-x`. Nếu `x` khác 0 thì bit dấu chắc chắn bằng 1, còn nếu `x` bằng 0 thì bit dấu bằng 0.

  Biểu thức `x | (~x + 1)` (tương đương $x \mid -x$) sẽ luôn có bit dấu (bit 31) bằng `1` với mọi $x \neq 0$, và chỉ có bit dấu bằng `0` duy nhất khi $x = 0$.

- Dịch phải số học 31 bit: `(x | (~x + 1)) >> 31`:
  - Khi $x = 0$: $0 \gg 31 = 0$.
  - Khi $x \neq 0$: Bit dấu là `1`, dịch phải số học sẽ lấp đầy 32 bit bởi bit `1`, tạo thành giá trị `-1` (`0xFFFFFFFF`).
- Cộng thêm `1`:
  - Khi $x = 0$: $0 + 1 = 1$.
  - Khi $x \neq 0$: $-1 + 1 = 0$.
- Thu được chính xác hành vi của phép `!x` mà không cần sử dụng toán tử `!`.

Mấu chốt của bài này là không cần kiểm tra toàn bộ 32 bit của `x`. Ta chỉ cần tìm cách biến điều kiện `x ≠ 0` thành thông tin ở bit dấu, sau đó dùng phép dịch phải để lấy thông tin đó và đảo kết quả về đúng dạng `!x`.

Mã nguồn

```
int logicNot(int x) {
  x = (x | (~x + 1)) >> 31;
  x = x + 1;
  return x;
}
```

```
69  // 2.5
70  int logicNot(int x) {
71      x = (x | (~x + 1)) >> 31;
72      x = x + 1;
73      return x;
74  }
```

```
[Running] cd "/home/DntdToM/scripts/ThucHanhLTHT/Week01/" && gcc source.c -o source && "/home/DntdToM/scripts/ThucHanhLTHT/Week01/"source
1.1 negative        Pass.
1.2 cal100x         Pass.
1.3 flipByte        Pass.
1.4 getnbit         Failed.
1.5 round2n         Failed.
2.1 isSameSign      Pass.
2.2 isPositive      Pass.
2.3 isMulpw2        Pass.
2.4 isLess2n        Pass.
2.5 logicNot        Pass.

Your score: 7.0
[Done] exited with code=0 in 0.456 seconds
```

B. TÀI LIỆU ĐÍNH KÈM

Link Repo của Lab1 (Github):
https://github.com/DntdToM/-NT209.R1.ANTN-.Group_10